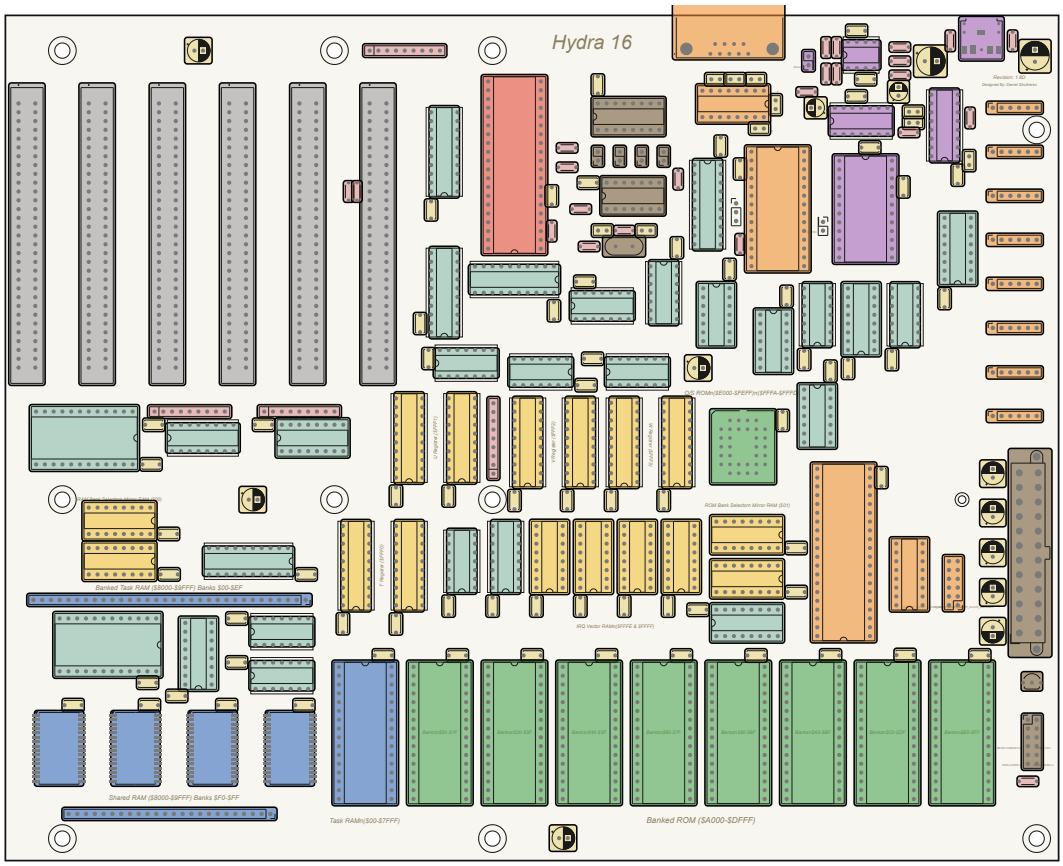
THE HYDRA PROJECT

HYDRA-16

Technical Reference



Every keyword, word, form, command and system call:
its syntax, its exact behaviour, its limits and its errors

HydraOS 1.0 · preview

About This Reference

This is the *Hydra-16 Technical Reference*, **in progress**. Its first chapter, HyBASIC's keywords, is being written letter by letter; the other chapters have one or two entries each so far, to show what they will be.

The *User's and Builder's Manual* teaches the Hydra, and its appendices list what each language has in a line apiece. This book is where to look when a line isn't enough: when the question is what a keyword does with an empty string, which errors a call can return, or what is left in a variable after a loop ends. It doesn't teach. It assumes the manual has been read, and it says the same thing about every entry, in the same order.

What it will hold

CHAPTER	ENTRIES	WRITTEN SO FAR
1. HyBASIC keywords	Every statement, function and operator	`ABS` to `CVN` (all of A to C); `FOR`…`NEXT`, `INPUT`, `OPEN`
2. Commands	The shell's own commands, and the programs in `/bin`	`cp`
3. System calls	Every call a program can make	`OPEN`, `READ`
4. HyForth words	The core and every library	`>number`
5. hylang forms	The special forms and the functions	`let`

The other reference appendices of the manual (the 65C02's instructions, the character sets, the memory map, the errors) will follow in the same way.

How to read an entry

Every entry has the same parts, in this order. A part with nothing to say is left out.

`Syntax`
: Each form the thing can take. Words in `CAPITALS` (or, for a command or a HyForth word, as printed) are typed as they are; *italic* names stand for what you supply; `[ ]` encloses what can be left out; `{ | }` is a choice of one; `...` means the last item can repeat.

`Arguments`
: Each one: its type, its range, and what it means. For a system call, the registers in.

`Result`
: What a function returns, a word leaves on the stack, or a call leaves in the registers.

`Description`
: What happens, in the order it happens: what is evaluated and when, what is changed, where it stops.

Errors
: Each error it can raise, with its number and its message as the Hydra prints it, and what causes it.

Notes
: Limits, edge cases, and behaviour that differs from what another system's user would expect.

See also
: Related entries.

Example
: A transcript, taken from the Hydra. Every one in this book is replayed in the emulator when the book is built, so what is printed here is what the ROM does.

How it was written

Each entry was written from the interpreter's, the program's or the kernel's source, and each statement of behaviour in it was then tried on the system. Where the source and the trial disagreed with what one would expect of the language elsewhere (QuickBASIC, the Forth standard), the entry says what the Hydra does.

Section and chapter numbers such as "section 13.2" and "Appendix G" are the *User's and Builder's Manual*'s.

Contents

About This Reference **2**

1 HyBASIC Keywords **6**

ABS 6
ABSOLUTE 7
AND 7
APPEND 8
ARG$ 8
AS 9
ASC 9
ASM ... END ASM 10
ATN 12
BANK 13
BASE 14
BASE$ 15
BEEP 15
BINARY 16
BIT 16
BYE 16
CALL 17
CASE 18
CDBL, CSNG 18
CHDIR 19
CHR$ 19
CINT, CLNG 20
CIRCLE 21
CLEAR 22
CLOSE 23
CLS 23
COLOR 24
COMMAND$ 24
COMPLEX 25
CONST 26
CONT 27
COS 28
CSRLIN 28
CVN 29
FOR ... NEXT 30
INPUT 31
OPEN 33

2 Commands **36**

cp 36

3 System Calls **39**

OPEN 39

READ 41

4 HyForth Words 44

>number 44

5 hylang Forms 46

let 46

CHAPTER 1

HyBASIC Keywords

HyBASIC compiles a program whole before it runs it: every line is read, every name resolved and every block matched, and only then does the first statement execute. An error the compiler finds (a `FOR` with no `NEXT`, a label that isn't there) is therefore reported before anything has run, with the line it was found on. A line typed at the prompt without a number is compiled and run the same way, as a program of one line.

An error is reported as its message, in small letters. `ERR` then holds its number, `ERR$` the message and `ERL` the line; the numbers given in each entry are `ERR`'s. A failure reported by the system (a disk that is full, a name that isn't there) has HyBASIC's own number where there is one that fits (53, file not found), and otherwise 256 plus the system's code (Appendix G).

A number is an integer of any length, a rational, a decimal or a complex number, and arithmetic on them is exact (Chapter 18). A string is a sequence of bytes, each 0 to 255, up to 64K of them.

ABS

A numeric function: the absolute value of a real number.

SYNTAX

```
ABS(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an integer, a rational or a decimal.

RESULT

x if *x* is zero or more, and *–x* if it is less. The result is the same kind of number as *x*, and exact: an integer gives an integer, of whatever length; a rational gives a rational in lowest terms; a decimal gives a decimal with the same digits. There is no largest value and no overflow.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex. A complex number has no sign, and `ABS` does not give its modulus
13	`type mismatch`	*x* is a string
2	`syntax error`	No argument, or more than one

NOTES

For the modulus of a complex number *z*, use `SQR(REAL(z) ^ 2 + IMAG(z) ^ 2)`.

SEE ALSO

`SGN`, `INT`, `FIX`, `REAL`, `IMAG`.

EXAMPLE

```
> PRINT ABS(-7); ABS(7); ABS(-7/2); ABS(-2.5)
 7  7  7/2  2.5
> PRINT ABS(-2 ^ 100)
 1267650600228229401496703205376
> PRINT ABS(3+4i)
illegal function call
> PRINT ERR; SQR(REAL(3+4i) ^ 2 + IMAG(3+4i) ^ 2)
 5  5
```

ABSOLUTE

Part of `CALL ABSOLUTE`, which calls machine code at an address. See `CALL`.

AND

An operator: the bitwise AND of two integers, which is also the logical AND of two truth values.

SYNTAX

```
a AND b
```

ARGUMENTS

a, b

Numeric expressions whose values are real.

RESULT

An integer. Each bit of it is 1 where the same bit is 1 in both *a* and *b*, and 0 elsewhere.

DESCRIPTION

Both operands are evaluated, always: there is no short cut when the first is 0. Each is then rounded to the nearest integer, a half going to the even one (1.5 and 2.5 both become 2), and the two integers are combined bit by bit.

The integers can be of any length. A negative one is taken in two's complement with as many sign bits as are needed, so –1 is all ones, and `-1 AND x` is *x* for every integer *x*.

A comparison gives –1 for true and 0 for false. Since –1 is all ones and 0 is none, `AND` of two comparisons is their logical AND, and `IF a > 0 AND b > 0 THEN` means what it says. `AND` of a truth value and some other number is a bit operation still: `(1 = 1) AND 2` is 2, which counts as true, but `1 AND 2` is 0, which is false, though both of its operands count as true.

`AND` binds more tighter than `OR`, `XOR`, `EQV` and `IMP`, and less tightly than `NOT` and the comparisons: `NOT a AND b = c OR d` is `((NOT a) AND (b = c)) OR d`.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An operand is a string
5	`illegal function call`	An operand is complex

SEE ALSO

`OR`, `XOR`, `NOT`, `EQV`, `IMP`, `BIT`, `SHL`.

EXAMPLE

```
> PRINT 12 AND 10; -1 AND 255; (1 < 2) AND (3 < 2); 1 AND 2
 8  255  0  0
> PRINT 2.5 AND 3; 1.5 AND 3; 2 ^ 40 + 5 AND 2 ^ 40 + 3
 2  2  1099511627777
> PRINT 0 AND (1 / 0)
division by zero
```

APPEND

A mode of `OPEN`: `OPEN path$ FOR APPEND AS #f` opens a file for writing at its end. See `OPEN`.

ARG$

A string function: one of the arguments the program was started with.

SYNTAX

```
ARG$(n)
```

ARGUMENTS

n

A numeric expression: which argument. 0 is the program's own name, 1 the first argument after it.

RESULT

The argument, as a string. The arguments are those of the command that started HyBASIC with a program to run: after `basic prog.bas one two`, `ARG$(0)` is `prog.bas`, `ARG$(1)` is `one` and `ARG$(2)` is `two`. An argument is passed whole, as the shell made it: one that was quoted there can hold spaces.

The empty string if there is no argument *n*: past the last one, for a negative *n*, and for every *n* when HyBASIC was started without a program (at its prompt).

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*n* is a string

NOTES

There is no count of the arguments. Read them until one is empty, or split `COMMAND$`.

SEE ALSO

`COMMAND$`, `ENVIRON$`.

EXAMPLE

```
% echo 'PRINT "["; ARG$(0); "]["; ARG$(1); "]["; ARG$(2); "]["; ARG$(3); "]"' >args.bas
% basic args.bas one 'two words'
[args.bas][one][two words][]
```

AS

Part of four statements, where it introduces a name, a number or a type: `DIM v AS type`, `OPEN path$ FOR mode AS #f`, `NAME old$ AS new$`, and a field's line in `TYPE`. See those.

ASC

A numeric function: the code of a string's first byte.

SYNTAX

```
ASC(s$)
```

ARGUMENTS

s$

A string expression of one byte or more.

RESULT

An integer from 0 to 255: the value of the first byte of *s$*. The rest of the string is not looked at. A string holds bytes, not characters of any one set, so the result is the byte itself; which character the console draws for it depends on the character set in use (Appendix E). The codes 32 to 126 are ASCII's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*s$* is the empty string
13	`type mismatch`	The argument is a number

NOTES

`ASC` is the inverse of `CHR$` for one byte: `ASC(CHR$(n))` is *n* for every *n* from 0 to 255, including 0 (a zero byte doesn't end a string). To test a string that may be empty, test `LEN(s$)` first.

SEE ALSO

`CHR$`, `LEN`, `MID$`, `VAL`.

EXAMPLE

```
> PRINT ASC("A"); ASC("Abc"); ASC(" "); ASC(CHR$(0) + "x"); ASC(CHR$(255))
 65  65  32  0  255
> PRINT ASC("")
illegal function call
```

ASM ... END ASM

A block: 65C02 assembly language inside a program, assembled when the program is compiled.

SYNTAX

```
ASM
    assembly language lines
END ASM

CALL ASM label [, a [, x [, y]]]
```

ARGUMENTS

label

A label defined in one of the program's `ASM` blocks, spelled as it is there. The assembler's names keep their case: `Add1` and `add1` are two labels.

a, x, y

Numeric expressions: the values the A, X and Y registers hold as the routine is entered, a byte each. Each is 0 if it is left out.

DESCRIPTION

`ASM` alone on a line starts a block and `END ASM` alone on a line ends it. The lines between are not HyBASIC: they are in the language of the `as` assembler (Chapter 27), with its labels, directives and expressions. A program can have several blocks; they are assembled together, as one source, in the order they appear, so a label in one can be used in another.

The blocks are assembled when the program is compiled, before it runs. Their code goes into a RAM bank of its own, 8K long, which is at `$8000` to `$9FFF` while the code runs. Statements don't run "through" a block: when control reaches `ASM`, it goes on after `END ASM`.

`CALL ASM label` calls the code at *label* as a subroutine (`jsr`), which returns with `rts`. `RREG` then gives the registers it returned with.

The assembler is given some of the program's names as symbols. A global numeric variable's name is the address of its value: five bytes, the first its kind (0 for an integer) and the next four an integer's value, low byte first. An integer `CONST`'s name is its value. Each is defined in capitals and in small letters. The names `A`, `X` and `Y` are left out, since they are the registers'.

ERRORS

ERR	MESSAGE	CAUSE
90	`ASM:` and the assembler's message	An error in a block, reported at its line when the program is compiled
90	`ASM:` *label*`: no such label in an ASM block`	`CALL ASM` names a label that no block defines

NOTES

The code's bank is at `$8000` only while the code runs, so the code can't use `BANK`'s bank at the same time, and a `PEEK` of `$8000` from HyBASIC doesn't see the code.

A program can name at most 64 different labels in its `CALL ASM` statements.

The code runs as part of HyBASIC's task, with the whole machine open to it. A routine that doesn't return, or that writes where it shouldn't, takes HyBASIC down with it.

SEE ALSO

`CALL`, `SYS`, `RREG`, `PEEK`, `POKE`, `BANK`.

EXAMPLE

```
> 10 CALL ASM seven : RREG A : PRINT A
> 20 CALL ASM add1, 41 : RREG A : PRINT A
> 30 END
> 40 ASM
> 50 seven: lda #7
> 60        rts
> 70 add1:  inc a
> 80        rts
> 90 END ASM
> RUN
 7
 42
```

ATN

A numeric function: the arctangent of a real number.

SYNTAX

```
ATN(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The angle, in radians, whose tangent is *x*: more than $-\pi/2$ and less than $\pi/2$. `ATN(0)` is the integer 0. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed), whatever kind of number *x* is.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

`4 * ATN(1)` is the usual way to get π in a BASIC, but here it multiplies the rounding of `ATN(1)` by four. `PI` gives π to the digits set, directly.

SEE ALSO

`TAN`, `SIN`, `COS`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT ATN(1); ATN(0); ATN(-1); ATN(1/2)
 0.785398163397  0 -0.785398163397  0.463647609001
> DIGITS 30 : PRINT ATN(1) : DIGITS 12
 0.785398163397448309615660845820
```

BANK

A statement and a function: chooses, or reports, the RAM bank that `PEEK` and `POKE` see at `$8000`.

SYNTAX

```
BANK n
BANK()
```

ARGUMENTS

n

A numeric expression from 0 to 255: a RAM bank's number.

RESULT

`BANK()` is the number last set by `BANK`, or 0 if none has been.

DESCRIPTION

The addresses `$8000` to `$9FFF` are a window, 8K wide, onto one RAM bank at a time (Appendix F). `BANK n` sets the bank that a `PEEK` or `POKE` of an address in that range will use. It changes nothing else: the bank is put in the window for the moment of each `PEEK` and `POKE` and then taken out again, because HyBASIC keeps its own data in banks and uses the window itself.

HyBASIC takes RAM banks for its own use as it needs them, for the program's text, its strings and its arrays, and `BANK` doesn't check for them: a `POKE` into a bank HyBASIC is using damages it. A program that wants a bank to itself should take one high in the task's range, which HyBASIC reaches last if at all.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 0 or more than 255
13	`type mismatch`	*n* is a string

NOTES

n is not checked against the banks the task has. A task with three RAM modules has banks 0 to 47.

`BANK` has no effect on the bank an `ASM` block's code runs in.

SEE ALSO

`PEEK`, `POKE`, `SYS`, `ASM`.

EXAMPLE

```
> PRINT BANK() : BANK 40 : POKE &H8000, 65 : PRINT PEEK(&H8000); BANK()
 0
 65  40
> BANK 41 : POKE &H8000, 66 : BANK 40 : PRINT PEEK(&H8000)
 65
```

BASE

A statement: sets the base that numbers are written and read in.

SYNTAX

```
BASE b$
```

ARGUMENTS

b$

A string expression: a base string (sections 18.4 to 18.6). `"d"` is decimal, `"x"` hexadecimal, `"b"` binary, `"o"` octal; `"7r"` is base 7.

DESCRIPTION

From the next statement on, every number that HyBASIC converts to text is written in the base, and every number it converts from text is read in it. That covers `PRINT`, `WRITE`, `STR$` and `PRINT #`; `INPUT`, `READ` and `VAL`; and the numbers written in the program's own lines that are compiled after the statement has run, which at the prompt means the lines typed after it.

A number with a prefix says its own base whatever the setting: `&HFF` and `&B1010` are 255 and 10 in every base.

The setting stays until another `BASE`, and is `"d"` when HyBASIC starts.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*b$* isn't a base string
13	`type mismatch`	*b$* is a number

NOTES

`OPTION BASE`, which sets the lowest index of arrays, is a different statement: see `OPTION`.

To write one number in another base without changing the setting, use `HEX$`, `OCT$`, or `STR$` with a base string.

SEE ALSO

`BASE$`, `DIGITS`, `HEX$`, `OCT$`, `STR$`, `VAL`.

EXAMPLE

On the second line the 10 and the 6 are read in hex, as sixteen and six, and their sum, twenty-two, is written in hex:

```
> X = 255 : BASE "x" : PRINT X
 FF
> PRINT 10 + 6; BASE$
 16 x
> BASE "d" : PRINT X
 255
```

BASE$

A string function: the base string that `BASE` last set.

SYNTAX

```
BASE$
```

RESULT

The string given to the last `BASE` statement, as it was given; `d` if there has been none.

SEE ALSO

`BASE`.

EXAMPLE

```
> PRINT BASE$ : BASE "16r" : PRINT BASE$ : BASE "d"
d
16r
```

BEEP

A statement: sounds the terminal's bell.

SYNTAX

```
BEEP
```

DESCRIPTION

Writes the byte 7 (BEL) to standard output. A terminal sounds its bell, or flashes its window, as it is set to. If output has been sent to a file, the byte goes into the file.

SEE ALSO

`SOUND`, `PLAY`.

BINARY

A mode of `OPEN`: `OPEN path$ FOR BINARY AS #f` opens a file for reading and writing bytes at any position. See `OPEN`, `GET` and `PUT`.

BIT

A numeric function: one bit of an integer.

SYNTAX

```
BIT(x, n)
```

ARGUMENTS

x

A numeric expression whose value is real. It is rounded to the nearest integer, a half going to the even one.

n

A numeric expression: which bit. Bit 0 is the lowest, worth 1; bit *n* is worth 2 to the power *n*.

RESULT

1 if bit *n* of *x* is set, and 0 if it is clear.

A negative *x* is taken in two's complement with endless sign bits, so every bit of −1 is 1, however high. 0 if *n* is negative.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

SEE ALSO

`AND`, `OR`, `XOR`, `SHL`, `SHR`.

EXAMPLE

```
> PRINT BIT(5, 0); BIT(5, 1); BIT(5, 2); BIT(-1, 100); BIT(2 ^ 70, 70)
 1  0  1  1  1
```

BYE

A statement: leaves HyBASIC. `BYE [n]` is another spelling of `SYSTEM [n]`. See `SYSTEM`.

CALL

A statement: calls a `SUB`, machine code at an address, or a label in an `ASM` block.

SYNTAX

```
[CALL] name [(] [argument [, argument] ...] [)]
CALL ABSOLUTE (address)
CALL ASM label [, a [, x [, y]]]
```

ARGUMENTS

name

The name of a `SUB` defined anywhere in the program.

argument

An expression, one for each of the `SUB`'s parameters, of the parameter's type.

address

A numeric expression, 0 to 65,535: where the machine code is.

DESCRIPTION

A `SUB`. The word `CALL` is optional, and so are the parentheses round the arguments: `Show "x", 2`, `CALL Show("x", 2)` and `CALL Show "x", 2` are the same call. The `SUB` runs, and the program goes on with the statement after the call.

An argument that is a variable, or an array's element, is passed **by reference**: the parameter is another name for it, and an assignment to the parameter changes it. Any other expression is passed by value. To pass a variable by value, put it in parentheses of its own, which makes it an expression: `Inc (X)`.

`CALL ABSOLUTE` calls the machine code at *address* as a subroutine, with the A, X and Y registers 0. It is `SYS address` by QuickBASIC's name. `RREG` gives the registers afterwards. See `SYS`.

`CALL ASM` calls a label of an `ASM` block. See `ASM`.

ERRORS

ERR	MESSAGE	CAUSE
35	`subprogram not defined`	No `SUB` has that name. Found when the program is compiled
37	`argument-count mismatch`	More arguments, or fewer, than the `SUB` has parameters
13	`type mismatch`	A string where the parameter is a number, or a number where it is a string

NOTES

A `SUB` can be called before the line that defines it: the whole program is compiled before any of it runs.

A `FUNCTION` is not called with `CALL`. It is used in an expression.

SEE ALSO

`SUB`, `FUNCTION`, `DECLARE`, `SYS`, `ASM`, `RREG`.

EXAMPLE

```
> 10 SUB Inc (N) : N = N + 1 : END SUB
> 20 X = 1 : Inc X : PRINT X
> 30 CALL Inc((X)) : PRINT X
> 40 DIM A(3) : A(1) = 5 : CALL Inc(A(1)) : PRINT A(1)
> RUN
 2
 2
 6
```

CASE

Part of `SELECT CASE`: `CASE` starts each of its choices. See `SELECT`.

CDBL, CSNG

Numeric functions: their argument, unchanged.

SYNTAX

```
CDBL(x)
CSNG(x)
```

ARGUMENTS

x

A numeric expression.

RESULT

x. In QuickBASIC these convert to double and to single precision. HyBASIC has one kind of numeric variable, which holds any number exactly, so there is nothing to convert: a rational stays a rational, and no digits are lost or gained. They are here so that programs written for QuickBASIC run.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

SEE ALSO

`CINT`, `FIXED`, `RATIONAL`, `DIGITS`.

EXAMPLE

```
> PRINT CDBL(1/3); CSNG(2.5); CDBL(7)
 1/3  2.5  7
```

CHDIR

A statement: changes the current directory.

SYNTAX

```
CHDIR path$
```

ARGUMENTS

path$

A string expression: the directory's name. One that doesn't start with `/` is taken from the current directory.

DESCRIPTION

Sets the current directory of HyBASIC's task, which is where a file name that doesn't start with `/` is looked for by `OPEN`, `LOAD`, `SAVE`, `FILES`, `KILL` and the rest. The change lasts until the next `CHDIR` or until HyBASIC ends. It doesn't change the directory of the shell that started HyBASIC.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name
76	`path not found`	The name is a file, not a directory
13	`type mismatch`	*path$* is a number

SEE ALSO

`DIR$`, `FILES`, `MKDIR`, `RMDIR`, `SHELL`.

EXAMPLE

```
> CHDIR "/ram" : OPEN "note.txt" FOR OUTPUT AS #1 : CLOSE : FILES
bin/            lib/            note.txt
> CHDIR "/nothere"
file not found
> CHDIR "/rom/bin/ls"
path not found
```

CHR$

A string function: a string of one byte.

SYNTAX

```
CHR$(n)
```

ARGUMENTS

n

A numeric expression from 0 to 255. One that isn't an integer is rounded to the nearest, a half going to the even one.

RESULT

A string one byte long, whose byte is *n*. Any value can be made, 0 included, and a string can hold any of them anywhere: `LEN(CHR$(0))` is 1.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 0 or more than 255
13	`type mismatch`	*n* is a string

NOTES

What the console shows for a byte depends on the byte and on the character set (Appendix E). The codes below 32 are controls to a terminal: `CHR$(7)` rings its bell, `CHR$(10)` starts a new line, and `CHR$(27)` begins an escape sequence (section 13.7).

SEE ALSO

`ASC`, `STRING$`, `SPACE$`.

EXAMPLE

```
> PRINT CHR$(72) + CHR$(105); LEN(CHR$(0)); ASC(CHR$(65.5)); ASC(CHR$(66.5))
Hi 1  66  66
> PRINT CHR$(256)
illegal function call
```

CINT, CLNG

Numeric functions: a real number rounded to the nearest integer.

SYNTAX

```
CINT(x)
CLNG(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The integer nearest to *x*. A value exactly half way between two integers goes to the even one: 2.5 gives 2, 3.5 gives 4, and −2.5 gives −2. This is the rounding QuickBASIC's `CINT` does.

The two functions are the same. An integer here can be of any length, so there is no range to pass, where QuickBASIC's `CINT` overflows beyond 32,767.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`INT` (rounds down), `FIX` (rounds toward zero), `FIXED`.

EXAMPLE

```
> PRINT CINT(2.5); CINT(3.5); CINT(-2.5); CINT(7/2); CINT(2.4); CINT(-2.6)
 2  4 -2  4  2 -3
> PRINT CLNG(2 ^ 40 + 1/2)
 1099511627776
```

CIRCLE

A statement: draws a circle, or a filled disc, on the graphics screen.

SYNTAX

```
CIRCLE [STEP] (x, y), r [, c [, start [, end [, aspect [, F]]]]]
```

ARGUMENTS

x, y
: Numeric expressions: the centre. With `STEP`, they are an offset from the last point referenced.

r
: A numeric expression, 0 or more: the radius.

c
: A numeric expression, 0 to 255: the colour. A mode with fewer colours uses the number's low bits: `SCREEN 1` has four colours, so 5 draws in colour 1. If it is left out, the pen's colour, which `COLOR` sets.

start, end, aspect
: Accepted, evaluated and ignored. QuickBASIC draws arcs and ellipses with them; HyBASIC always draws a whole circle.

F
: The letter F: fill the circle, making a disc.

DESCRIPTION

Needs a graphics mode (`SCREEN` 1 or more). The coordinates and the radius are in pixels, each rounded to the nearest, unless `WINDOW` has set coordinates of the program's own; then the radius is in the window's x units. The centre becomes the last point referenced, where a following `STEP` or `LINE -(x, y)` starts

from.

A radius of 0 draws the centre's one dot.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or *c* is not 0 to 255
258	`invalid argument`	The driver refused it: a negative radius, or a centre far off the bitmap
13	`type mismatch`	An argument is a string

SEE ALSO

`SCREEN`, `COLOR`, `PSET`, `LINE`, `PAINT`, `WINDOW`, `POINT`.

EXAMPLE

With a Vera X card. (`SCREEN` clears the console, so this one is shown as it was typed, not as a replayed transcript.)

```
> SCREEN 13 : CIRCLE (100, 100), 20, 200
> PRINT POINT(120, 100); POINT(100, 80); POINT(100, 100)
 200  200  0
> CIRCLE (100, 100), 10, 3, , , , F : PRINT POINT(100, 100); POINT(105, 100)
 3  3
> SCREEN 0
```

CLEAR

A command of the prompt: forgets every variable.

SYNTAX

```
CLEAR
```

DESCRIPTION

Every variable and array of the program is dropped, so that each is 0, empty or not yet made when it is next used. The program's lines stay. A program that was stopped can no longer be continued with `CONT`.

`CLEAR` is one of the commands that work only when typed at the prompt, alone on its line. `RUN` clears the variables itself as it starts.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`CLEAR` in a program's line, or after other statements on a line typed at the prompt

SEE ALSO

`NEW`, `RUN`, `ERASE`, `CONT`.

CLOSE

A statement: closes files.

SYNTAX

```
CLOSE [[#]f [, [#]f] ...]
```

ARGUMENTS

f

A numeric expression: the number of a file that `OPEN` opened.

DESCRIPTION

Each file named is closed, and its number is free to be used by another `OPEN`. With no numbers, every open file is closed.

Closing a number that is not open is not an error: nothing happens. That holds for any number, 0 and numbers above 255 included.

What a program has written with `PRINT #`, `WRITE #` or `PUT` is in the file as each statement finishes, not held back until `CLOSE`; closing only gives up the file.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*f* is a string

NOTES

`RUN` closes every file as it starts, and files are closed when HyBASIC ends.

SEE ALSO

`OPEN`, `FREEFILE`.

CLS

A statement: clears the screen.

SYNTAX

```
CLS
```

DESCRIPTION

Clears the console's text and puts the cursor at the top left. In a graphics mode (`SCREEN` 1 or more) the bitmap is cleared to colour 0 as well, and the last point referenced becomes its middle.

If output is not a console, the bytes that would clear one (an escape sequence, section 13.7) go where output goes.

SEE ALSO

LOCATE, SCREEN, COLOR.

COLOR

A statement: sets the colours of text, or in a graphics mode the colour drawn with.

SYNTAX

```
COLOR [fg] [, bg]
```

ARGUMENTS

fg

A numeric expression: the foreground colour.

bg

A numeric expression: the background colour.

DESCRIPTION

With no graphics mode (SCREEN 0), COLOR sets the colours of the text written from then on. Each is 0 to 15: 0 to 7 are the terminal's eight colours (black, red, green, yellow, blue, magenta, cyan, white), and 8 to 15 are their bright forms. One that is left out is not changed. The colours are set by writing an escape sequence (section 13.7), so what they look like is the terminal's choice.

In a graphics mode (SCREEN 1 or more), *fg* sets the pen: the colour used by the drawing statements when they are given none. *bg* is accepted and ignored.

ERRORS

ERR	MESSAGE	CAUSE
5	illegal function call	A text colour that isn't 0 to 15
13	type mismatch	An argument is a string

SEE ALSO

SCREEN, PALETTE, CLS, PSET, LINE, CIRCLE.

COMMAND$

A string function: the arguments the program was started with, as one string.

SYNTAX

```
COMMAND$
```

RESULT

The arguments after the program's name, joined with one space between each, 255 bytes at most. The empty string if there were none, or if HyBASIC was started without a program.

An argument that held spaces of its own can't be told from two arguments here. `ARG$` gives each one whole.

SEE ALSO

`ARG$`, `ENVIRON$`.

EXAMPLE

```
% echo 'PRINT "["; COMMAND$; "]"' >cmd.bas
% basic cmd.bas one 'two words'
[one two words]
% basic cmd.bas
[]
```

COMPLEX

A numeric function: a complex number made from its two parts.

SYNTAX

```
COMPLEX(re, im)
```

ARGUMENTS

re*, *im

Numeric expressions whose values are real: the real part and the imaginary part.

RESULT

The number *re* + *im* i. The parts keep their kinds: a rational real part with a decimal imaginary part is a complex number of exactly those. If *im* is 0 the result is the real number *re*, not a complex number with nothing in its imaginary part.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*re* or *im* is itself complex
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

NOTES

A complex number can also be written directly, as `1+2i` or `0.5i`: digits with an `i` after them are an imaginary number. The `i` must follow a number. `1+i` is 1 plus the variable `I`.

SEE ALSO

REAL, IMAG, SQR.

EXAMPLE

```
> PRINT COMPLEX(1, 2); COMPLEX(1/2, 0.5); COMPLEX(3, 0); COMPLEX(0, 1) ^ 2
 1+2i  1/2+0.5i  3 -1
> PRINT COMPLEX(1+2i, 3)
illegal function call
```

CONST

A statement: gives names to constant values.

SYNTAX

```
CONST name = value [, name = value] ...
```

ARGUMENTS

name

A name that has not been used: a string's if it ends in `$`, otherwise a number's.

value

A constant expression of the name's type: numbers and strings written out, other constants, and operators and built-in functions applied to them.

DESCRIPTION

The value is worked out when the program is compiled, and the name stands for it from then on. A constant can be used wherever a value can, and it is known in every `SUB` and `FUNCTION` without being `SHARED`.

A constant can't be assigned to, and its name can't be given to anything else.

ERRORS

ERR	MESSAGE	CAUSE
10	`duplicate definition`	*name* is already a constant or a variable; or an assignment to a constant
87	`not a constant`	*value* uses a variable, an array or a `FUNCTION`
13	`type mismatch`	A string for a numeric name, or a number for a string name

NOTES

An integer constant's name is also a symbol in the program's `ASM` blocks, where it stands for its value.

SEE ALSO

DIM, LET, ASM.

EXAMPLE

```
> CONST LIMIT = 5, UNIT$ = "cm", TWICE = LIMIT * 2 : PRINT LIMIT; UNIT$; TWICE
 5 cm 10
> LIMIT = 6
duplicate definition
> V = 3 : CONST LATER = V
not a constant
```

CONT

A command of the prompt: goes on with a program that was stopped.

SYNTAX

```
CONT
```

DESCRIPTION

A program stops, with its state kept, at a `STOP` statement or when CTRL-C is pressed. `CONT` carries on from the statement after the one it stopped at, with every variable as it is now: variables can be looked at, and changed, at the prompt in between.

A program can no longer be continued once it has ended, once one of its lines has been changed, added or removed, or after `NEW`, `LOAD` or `CLEAR`.

ERRORS

ERR	MESSAGE	CAUSE
17	`cannot continue`	No program is stopped
89	`only at the prompt`	`CONT` in a program's line

SEE ALSO

`STOP`, `END`, `RUN`.

EXAMPLE

```
> 10 PRINT "one" : STOP : PRINT "two"
> 20 PRINT "three"
> RUN
one
Break in line 10
> CONT
two
three
> CONT
cannot continue
```

COS

A numeric function: the cosine of a real number.

SYNTAX

```
COS(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an angle in radians.

RESULT

The cosine of *x*, from −1 to 1. `COS(0)` is the integer 1. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

An angle in degrees is turned into radians by multiplying by `PI / 180`.

`PI` is itself rounded to the digits set, so `COS(PI / 2)` is not exactly 0. It is a very small number: the cosine of the angle that `PI / 2` really is.

SEE ALSO

`SIN`, `TAN`, `ATN`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT COS(0); COS(1); COS(PI); COS(60 * PI / 180)
 1  0.540302305868 -1  0.5
```

CSRLIN

A numeric function: the row the cursor is on.

SYNTAX

```
CSRLIN
```

RESULT

The cursor's row, counted from 1 at the top of the screen. 0 if HyBASIC has no console (its input or output is a file or a pipe).

DESCRIPTION

HyBASIC does not keep track of the row. It asks the terminal each time, by writing a cursor position request (section 13.7) and reading the terminal's reply, so the answer is right whatever has been written to the screen, by this program or another.

NOTES

The column is `POS(0)`.

SEE ALSO

`POS`, `LOCATE`.

CVN

A numeric function: the number that a string of bytes holds, in the stored format.

SYNTAX

```
CVN(s$)
```

ARGUMENTS

s$

A string expression: a number's bytes, as `MKN$` makes them and as `PUT` writes a numeric variable to a file.

RESULT

The number. It is exactly the number that was stored, of the same kind: `CVN(MKN$(x))` is *x* for every number *x*.

DESCRIPTION

A number's stored form is as long as the number needs: a small integer takes one byte, a rational or a long integer more.

It replaces QuickBASIC's `CVI`, `CVL`, `CVS` and `CVD`, as `MKN$` replaces `MKI$` and the rest: there is one kind of number, so one pair of functions.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*s$* is not a number's stored form
13	`type mismatch`	*s$* is a number

NOTES

A string that was never a number may still be some number's stored form, and then there is no error: `CVN("x")` is –8.

SEE ALSO

`MKN$`, `GET`, `PUT`.

EXAMPLE

```
> PRINT CVN(MKN$(1/3)); CVN(MKN$(-7)); CVN(MKN$(2+3i)); LEN(MKN$(7)); LEN(MKN$(1/3))
 1/3 -7  2+3i  1  3
```

FOR ... NEXT

A statement block: a loop that counts a variable from one value to another.

SYNTAX

```
FOR v = start TO limit [STEP step]
    [statements]
NEXT [v [, v] ...]
```

ARGUMENTS

v

The loop's counter: a simple numeric variable. An array's element can't be one.

start, limit, step

Numeric expressions. *step* is 1 if `STEP` is left out.

DESCRIPTION

1. *start* is assigned to *v*, and *limit* and *step* are evaluated and kept. They are evaluated this once only: a variable they were worked out from can change inside the loop without changing them.
2. The test: if *step* is zero or more and *v* is greater than *limit*, or *step* is negative and *v* is less than *limit*, the loop ends, and the program goes on after its `NEXT`. The test is made before the first pass too, so the statements may not run at all; *v* then keeps the value *start*.
3. Otherwise the statements run.
4. At `NEXT`, *step* is added to *v*, and the loop goes back to the test at step 2.

When the loop ends at the test, *v* holds the first value that failed it: after `FOR I = 1 TO 3`, `I` is 4.

The addition is exact. A loop with a fractional step, `STEP 0.1` or `STEP 1/3`, reaches its limit on the last pass, where a binary floating-point BASIC may stop one short.

The statements may assign to *v*; `NEXT` adds *step* to whatever it then holds.

`EXIT FOR` leaves the innermost loop at once, with *v* as it is.

`NEXT` with no variable closes the innermost open loop. `NEXT v` does the same, and the compiler checks that *v* is that loop's counter. `NEXT j, i` is `NEXT j : NEXT i`.

ERRORS

ERR	MESSAGE	CAUSE
26	`FOR without NEXT`	A `FOR` whose `NEXT` isn't there. Found when the program is compiled: nothing has run
1	`NEXT without FOR`	A `NEXT` with no loop open, or naming a variable that isn't the innermost loop's counter
80	`block without its start or end`	The loop and another block overlap: a `NEXT` inside a one-line `IF`, say
13	`type mismatch`	*v* is a string variable, or *start*, *limit* or *step* a string
2	`syntax error`	*v* is not a simple variable

NOTES

With a *step* of 0 the counter never changes and the loop doesn't end by itself: only `EXIT FOR`, an error or CTRL-C ends it.

At the prompt, a line is compiled alone, so a loop typed there must be whole on its line: `FOR I = 1 TO 3` by itself is `FOR without NEXT`.

SEE ALSO

`EXIT`, `DO...LOOP`, `WHILE...WEND`.

EXAMPLE

```
> FOR I = 1 TO 3 : PRINT I; : NEXT : PRINT "then"; I
 1  2  3 then 4
> FOR I = 5 TO 1 : PRINT I; : NEXT : PRINT "then"; I
then 5
> FOR X = 0 TO 1 STEP 1/3 : PRINT X; : NEXT : PRINT
 0  1/3  2/3  1
> N = 3 : FOR I = 1 TO N : N = 10 : PRINT I; : NEXT : PRINT
 1  2  3
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : PRINT I; J; : NEXT J, I : PRINT
 1  1  1  2  2  1  2  2
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : NEXT I
NEXT without FOR
```

INPUT

A statement: reads a line and assigns its fields to variables. The line comes from the console, or from a file.

SYNTAX

```
INPUT [;] ["prompt" {; | ,}] v [, v] ...
INPUT #f, v [, v] ...
```

ARGUMENTS

prompt

A string in quotes, written before the line is read. It must be a literal: a string variable there is a syntax error. After it, `;` adds a question mark and a space, and `,` adds nothing. With no prompt at all, `?` is written.

v

A numeric or string variable, or an array's element. Each takes one field of the line.

f

The number of a file opened `FOR INPUT` or `FOR BINARY`.

DESCRIPTION

From the console. The prompt is written and one line is read, with the console's line editing (section 13.2). Only its first 255 bytes are used. The line is split into fields at its commas, and each variable takes a field, in order:

- A numeric variable's field is read as a number, in any form a program's own numbers may take: `12`, `-7/2`, `2.5`, `1e3`, `&HFF`. An empty field is 0.
- A string variable's field is the text up to the next comma, with the spaces before and after it dropped. A field that starts with a double quote runs to the closing quote instead, and can hold commas and spaces.

If the line has fewer fields than there are variables, or more, or a numeric field that isn't a number, HyBASIC writes `?Redo from start` and the statement starts again from its prompt. It is never an error.

From a file. One line of file *f* is read, and split and assigned in the same way, with three differences. There is no prompt and nothing is asked again: a numeric field that isn't a number is 0. If the line runs out of fields before the variables do, the next line is read and its fields carry on. Fields left on the line after the last variable is filled are dropped: the next `INPUT #` starts at the next line.

The semicolon after `INPUT` matters only when input doesn't come from the console (`basic prog <answers`). The console echoes the Return that ends a line; a file doesn't, so HyBASIC writes a new line itself after reading one. `INPUT ;` leaves it out, and what is printed next follows the prompt on the same line.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	No line is left: in file *f*, or in standard input when it isn't the console
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`
2	`syntax error`	The prompt isn't a string literal

CTRL-C at the prompt stops the program, as it does anywhere else (`Break`).

NOTES

To read a whole line, commas and all, into one string, use `LINE INPUT`. To read keys without waiting for Return, use `INKEY$` or `INPUT$`.

SEE ALSO

`LINE INPUT`, `INPUT$`, `INKEY$`, `OPEN`, `EOF`, `READ`.

EXAMPLE

```
> 10 INPUT "Two numbers"; A, B
> 20 PRINT A + B
> 30 INPUT "Name, age: ", N$, G
> 40 PRINT "["; N$; "]"; G
> 50 INPUT X, Y$
> 60 PRINT X; "["; Y$; "]"
> RUN
Two numbers? 3
?Redo from start
Two numbers? 1/2, 2.5
 3
Name, age: Sam Smith , 12
[Sam Smith] 12
? , "x, y"
 0 [x, y]
```

Fields that carry on from one line of a file to the next:

```
> 10 OPEN "/ram/n.txt" FOR OUTPUT AS #1
> 20 PRINT #1, "1" : PRINT #1, "2, 3" : PRINT #1, "4"
> 30 CLOSE #1 : OPEN "/ram/n.txt" FOR INPUT AS #1
> 40 INPUT #1, A, B : PRINT A; B
> 50 INPUT #1, C : PRINT C
> 60 INPUT #1, D
> RUN
 1  2
 4
line 60: input past end of file
```

OPEN

A statement: opens a file, and gives it a number for the statements that read and write it.

SYNTAX

```
OPEN path$ FOR {INPUT | OUTPUT | APPEND | BINARY} AS [#]f [LEN = n]
```

ARGUMENTS

path$

A string expression: the file's name. A name that doesn't start with `/` is taken from the current directory (`CHDIR`). It can be any name in the program's namespace: a file on a disk, or a device such as

`/dev/cons`.

f

A numeric expression from 1 to 255: the number the file is known by until it is closed. The `#` is optional.

n

Accepted, evaluated and ignored. QuickBASIC's record length has no meaning here: `GET` and `PUT` take a position in bytes.

DESCRIPTION

The mode decides how the file is opened, and which statements may then use it:

MODE	A FILE THAT EXISTS	A FILE THAT DOESN'T	STARTS AT	MAY THEN BE USED BY
`INPUT`	Opened for reading	Error 53	Its first byte	`INPUT #`, `LINE INPUT #`, `INPUT$`, `EOF`, `LOF`, `LOC`, `SEEK`
`OUTPUT`	Emptied, and opened for writing	Made	Its first byte	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`APPEND`	Opened for writing, its contents kept	Made	Its end	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`BINARY`	Opened for reading and writing	Made	Its first byte	All of the above, and `GET` and `PUT`

Up to 8 files can be open at once. A file stays open until `CLOSE` names it, `CLOSE` with no number closes them all, or a `RUN` starts (which closes every file first). Leaving HyBASIC closes them too.

`EOF(f)` is always true (–1) for a file opened `FOR OUTPUT` or `FOR APPEND`.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	*f* is less than 1 or more than 255
55	`file already open`	Number *f* is in use
67	`too many files`	8 files are open already
53	`file not found`	`FOR INPUT`: there's no such file. Any mode: a directory in the path isn't there
70	`permission denied`	The file, or its disk (`/rom`), can't be written
13	`type mismatch`	*path$* is a number, or *f* a string

The system's other refusals come through as its own errors (a full disk is 61, `disk full`).

NOTES

The limit of 8 is HyBASIC's own. Each open file also takes one of the task's 16 file descriptors, three of which are the console's.

One file can be open under two numbers at once, each with its own position: opening it `FOR INPUT` as `#2` while it is open `FOR OUTPUT` as `#1` is allowed, and reads what has been written so far.

A file written with `PRINT #` holds exactly the bytes `PRINT` would have sent to the screen, each line ended by a line feed (byte 10).

SEE ALSO

`CLOSE`, `PRINT`, `WRITE`, `INPUT`, `LINE INPUT`, `GET`, `PUT`, `SEEK`, `EOF`, `LOF`, `LOC`, `KILL`, `NAME`.

EXAMPLE

```
> OPEN "/ram/log.txt" FOR OUTPUT AS #1 : PRINT #1, "one" : PRINT LOF(1); EOF(1) : CLOSE
#1
 4 -1
> OPEN "/ram/log.txt" FOR APPEND AS #1 : PRINT #1, "two" : PRINT LOF(1) : CLOSE #1
 8
> OPEN "/ram/log.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT L$; EOF(1)
one 0
> PRINT #1, "three"
bad file mode
> OPEN "/ram/other.txt" FOR OUTPUT AS #1
file already open
> CLOSE : OPEN "/ram/nothere.txt" FOR INPUT AS #1
file not found
> PRINT ERR
 53
```

CHAPTER 2

Commands

A command is a program in `/bin`, or one of the shell's own. Each ends with an **exit status**, which rc keeps in `$status`: empty for success, and a short text for failure. Most commands fail with the status `1`, and with `usage` when their arguments make no sense. A command's messages go to standard error, as `name: what: why`, so they aren't mixed into a pipe's data.

A path in an argument can be at most 63 characters long once it has been made absolute.

cp

A program (`/bin/cp`): copies files, and with `-r` directories.

SYNTAX

```
cp [-r] from to
cp [-r] from ... dir
```

ARGUMENTS

-r

Copy directories too: each one's whole tree.

from

A file to copy, or with `-r` a directory. There can be several if the last name is a directory.

to

The copy's name. If it is an existing directory, the copy goes into it under *from*'s own name.

dir

An existing directory. Each *from* is copied into it under its own name.

DESCRIPTION

Each *from* is handled in turn, and one that fails doesn't stop the rest.

A file is copied by opening it for reading, creating the destination, and reading and writing 512 bytes at a time until a read returns nothing. Creating the destination empties a file that is already there. The copy is a new file: it has its own date, the time of copying, and is not read-only even if the original is.

Before copying, `cp` compares the two files' device, disk and file identifier. If they are the same file, by whatever two names, it refuses to copy it onto itself, which would otherwise empty it.

Anything that can be opened and read can be a *from*, a device included: `cp /dev/time now` copies what one read of the clock gives.

Without `-r`, a *from* that is a directory is reported and skipped. With `-r`, the directory is made at the destination (one already there is used as it is), and then everything in it is copied in the same way, down to 16 directories deep.

EXIT STATUS

Empty if every file was copied. `1` if any was not. `usage`, after a line saying how `cp` is used, if there are fewer than two names or an option other than `-r`.

ERRORS

Each is printed as `cp: name: reason`, where *name* is the file that could not be read or written.

REASON	CAUSE
`not found`	*from* isn't there, or a directory in *to*'s path isn't
`is a directory`	*from* is a directory, and there's no `-r`
`not a directory`	Several *from*s, and the last name isn't a directory
`the same file`	*from* and *to* are one file
`read-only`	The destination's disk can't be written (`/rom`)
`name too long`	The destination's path would pass 63 characters

Any other reason is the system's own error message for what failed (Appendix G): a full disk, for one.

NOTES

If a read or a write fails part-way, the destination is left holding what was written up to then.

`cp` doesn't ask before it overwrites.

SEE ALSO

`mv`, `rm`, `mkdir`, `cat`, `cmp`.

EXAMPLE

```
% echo hello >a.txt
% cp a.txt b.txt; cat b.txt
hello
% mkdir d; cp a.txt b.txt d; ls d
a.txt
b.txt
% cp d e; echo $status
cp: d: is a directory
1
% cp -r d e; ls e
a.txt
b.txt
% cp a.txt a.txt
cp: a.txt: the same file
% cp a.txt /rom/x
cp: /rom/x: read-only
% cp a.txt; echo $status
usage: cp [-r] from to, or cp [-r] from ... dir
usage
```

CHAPTER 3

System Calls

A system call is a `jsr` to a fixed address in the jump table at `$F800`. Its arguments go in A, X, Y and the call registers r0 to r15 (two bytes each, in the zero page from `$02`); its results come back in the same places, a 16-bit result in A (low byte) and X (high byte). On return, C = 0 means success. C = 1 means failure, and A holds an error code (Appendix G). A call may change A, X, Y, the flags and r0 to r15, including registers it doesn't use for a result; it never changes the program's own zero page, `$22` to `$7F`.

A call that *can wait* may suspend the task until something happens elsewhere: a key, a disk, another task. A note delivered to the task ends such a wait, and the call returns `E_INTR`.

Each call's name is defined in `hydra.inc` (`/lib/as`), with the constants and error codes used here.

OPEN

System call `$F8C0`: opens a file that exists, and returns a file descriptor for it. It can wait.

SYNTAX

```
            LDR r0, path        ; r0 = the address of the path
            lda #mode
            jsr OPEN
            bcs failed          ; A = the error code
            sta fd              ; A = the file descriptor
```

ARGUMENTS

r0

The address of the path: its bytes, then a zero byte.

A

The mode: one of `O_READ` (0), `O_WRITE` (1) or `O_RDWR` (2), to which can be added `O_TRUNC` (`$10`), to empty the file as it is opened, and `O_NONBLOCK` (`$40`), to make reads and writes of this descriptor return `E_AGAIN` where they would otherwise wait.

RESULT

C = 0, and A = the file descriptor: a number from 0 to 15, the lowest one the task has free.

DESCRIPTION

1. The path is made absolute and clean. One that doesn't start with `/` (or with `#`, a device's own name) has the task's current directory put before it; then `.` elements and doubled slashes are removed, and each `..` removes the element before it. The result may be at most 63 bytes (`PATH_MAX`).
2. The path is looked up in the task's namespace (section 9.5). Where a directory is a union of several (section 9.7), each member is tried in turn, and the first that has the name is the one opened. A member whose device has gone is passed over as if it didn't have the name.
3. The device's driver is asked to open the file in the mode given. If it refuses, its error is the call's.
4. The descriptor is set to the file, at offset 0.

A descriptor has an offset of its own, which `READ`, `WRITE` and `SEEK` move. Descriptors made from it by `DUP`, and those a child task is given by `SPAWN`, share that offset; a second `OPEN` of the same file does not.

Opening a directory with `O_READ` succeeds: it reads as a sequence of 64-byte stat records, one for each file in it. When the directory is a union, the records of all its members follow one another.

`OPEN` never makes a file. `CREATE` does, and opens it.

ERRORS

CODE	NAME	CAUSE
`$20`	`E_NOENT`	No such file, in any member of a union
`$2B`	`E_NODEV`	The path names a device that isn't there
`$0A`	`E_NAMETOOLONG`	The path, made absolute, is longer than 63 bytes
`$26`	`E_MFILE`	The task has all 16 of its descriptors in use
`$27`	`E_NFILE`	All 48 of the system's channels are in use (every open file in every task takes one; `DUP`s share)
`$05`	`E_INTR`	A note arrived while the call was waiting for the driver

A driver adds its own: `E_ROFS` (`$29`) for a write mode on a read-only disk, and `E_ISDIR` (`$23`) for a write mode on a directory.

NOTES

Descriptors 0, 1 and 2 are standard input, output and error, already open when a program starts. A task that closes one of them will get that number from its next `OPEN`.

A descriptor that isn't closed is closed when the task ends.

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-open` (*z-addr mode -- fd ior*), the path zero-terminated (`>z` makes one)
hylang	`(sys-open path [mode])`: the descriptor. A failure is an error, its text the path and the system's message
HyBASIC	`SYS "OPEN"`; or, for most purposes, the `OPEN` statement (Chapter 1)

SEE ALSO

`CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `DUP`, `FSTAT`.

EXAMPLE

From hylang. Descriptors 0 to 2 are in use, so the first file opened is 3:

```
hylang> (sys-open "/dev/time" 0)
=> 3
hylang> (sys-open "/dev/nothing" 0)
=> Error: /dev/nothing: not found
hylang> (sys-open "/rom/bin/ls" 1)
=> Error: /rom/bin/ls: read-only
hylang> (sys-open "/ram" 1)
=> Error: /ram: is a directory
hylang> (sys-open "#q/x" 0)
=> Error: #q/x: no such device
```

READ

System call `$F8C9`: reads bytes from an open file into memory. It can wait.

SYNTAX

```
            LDR r0, buf         ; r0 = where the bytes go
            LDR r1, count       ; r1 = how many, at most
            lda fd
            jsr READ
            bcs failed          ; A = the error code
                                ; A/X = how many were read; 0 is the end of the file
```

ARGUMENTS

A

A file descriptor, open with `O_READ` or `O_RDWR`.

r0

The address of a buffer in the task's memory.

r1

The most bytes to read: 0 to 65,535. The buffer must have room for them.

RESULT

C = 0, and A/X = the number of bytes read, which were put in the buffer from its start. The descriptor's offset has moved on by that many.

DESCRIPTION

The kernel asks the file's driver for the bytes at the descriptor's offset, at most 512 at a time (`IO_UNIT`), and repeats until one of three things happens:

- **The count is reached.** The result equals r1.
- **The file ends.** The result is what there was before the end; a read that starts at the end returns 0. A result of 0 is how the end of a file is recognised, and it is not an error.
- **The driver returns fewer bytes than it was asked for.** The kernel stops there and returns them. This is how a device says "that is all for now": the console returns one line, however large the buffer.

A result smaller than r1 therefore does not by itself mean the end of the file. A program that needs exactly *n* bytes must call `READ` again for the remainder until it has them or gets 0.

When the driver has nothing yet (no key typed, an empty pipe), the task waits. With `O_NONBLOCK` set on the descriptor, the call returns `E_AGAIN` instead.

A directory reads as 64-byte stat records (`SR_SIZE`), one for each file in it: ask for a multiple of 64.

ERRORS

CODE	NAME	CAUSE
`$25`	`E_BADF`	A isn't an open descriptor, or it was opened with `O_WRITE`
`$04`	`E_AGAIN`	Nothing to read yet, and the descriptor is non-blocking
`$05`	`E_INTR`	A note arrived while the call was waiting

A driver adds its own: a disk that fails answers `E_IO` (`$2A`).

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-read` (*fd c-addr u -- u2 ior*): *u2* bytes were read
hylang	`(sys-read fd count)` : the bytes read, as a string
HyBASIC	`SYS "READ"`; or `INPUT #`, `LINE INPUT #`, `INPUT$` and `GET`

SEE ALSO

`OPEN`, `WRITE`, `SEEK`, `CLOSE`, `PIPE`.

EXAMPLE

The offset moving on, a count cut short by the end of the file, and 0 at the end:

```
hylang> (set {fd} (sys-open "/ram/t" 0))
=> NIL
hylang> (sys-read fd 5)
=> "hello"
hylang> (sys-read fd 100)
=> " world\n"
hylang> (sys-read fd 100)
=> ""
hylang> (sys-close fd)
=> NIL
```

CHAPTER 4

HyForth Words

A cell is 16 bits. A double is two cells, 32 bits, its high cell on top of the stack. A flag is 0 for false and –1 (all bits set) for true. *ior* is 0 for success and a non-zero error code otherwise. Stack pictures follow the Forth standard's notation: *n* signed, *u* unsigned, *d* and *ud* doubles, *c-addr u* the address and length of a string.

Each entry says which word set a word belongs to, and whether it is in the ROM core or needs a library loaded with `lib` first. Where a word is the standard's, the entry describes what HyForth does in the cases the standard leaves open.

>number

Core word set, in the ROM core. Converts the digits at the start of a string, adding them to a double.

SYNTAX

```
>number  ( ud1 c-addr1 u1 -- ud2 c-addr2 u2 )
```

ARGUMENTS

ud1

The number so far: 0 to start a new one.

c-addr1 u1

The text to convert.

RESULT

ud2

ud1 with each converted digit added: for every digit, *ud* × `base` + the digit's value.

c-addr2 u2

What was not converted: the address of the first character that isn't a digit and the number of characters from there to the end. *u2* is 0 if the whole string was digits, and *c-addr2* is then the address just past it.

DESCRIPTION

Characters are taken from the left for as long as each is a digit in the current `base`. The digits are `0` to `9`, then the letters `A` to `Z` for 10 to 35; small letters count as capitals. A character whose value is not less than `base` is not a digit, and conversion stops at it: in decimal `a` stops it, in hex `g` does.

Nothing else is understood. A sign, a prefix (`$`, `#` or `%`) and a decimal point all stop the conversion like any other non-digit; an empty string converts nothing and returns *ud1* unchanged. Signs and prefixes are the text interpreter's business: it deals with them itself, and calls this word's code for the digits.

ERRORS

None. A string with no digits is not an error: *ud2* = *ud1* and *u2* = *u1*.

NOTES

A value that passes 32 bits wraps: only its low 32 bits are kept, and no error is raised. (The standard calls this an ambiguous condition.) To read a number of any length, use the numbers library (`lib numbers`).

`base` can be 2 to 36.

SEE ALSO

`base`, `decimal`, `hex`, `s>d`, `evaluate`.

EXAMPLE

`nip` drops the address, leaving the double (its low cell first, as `.s` shows it) and the count of characters not converted:

```
/> 0 0 s" 123abc" >number nip .s
<3> 123 0 3
/> drop 2drop
/> hex 0 0 s" ff zz" >number nip .s decimal
<3> FF 0 3
/> drop 2drop
/> 0 0 s" -5" >number nip .s
<3> 0 0 2
/> drop 2drop
/> 0 0 s" 99999999999" >number nip .s
<3> 2839 -20518 0
```

CHAPTER 5

hylang Forms

An expression in parentheses is evaluated: its first element names a function or a special form, and the rest are its arguments. A function's arguments are evaluated before it is called, left to right. A **special form** receives its arguments as they were written, and decides for itself which to evaluate and when; each entry for one says exactly that. An expression in braces, a Q-expression, is data: it is not evaluated until something runs it.

A name is looked up in the current **scope**, then in the scope that encloses it, and so on out to the global scope. Each call of a function has a scope of its own, enclosed by the scope the function was made in.

An error is a value: `Error:` and a message. An expression that is given an error as an argument returns it at once, so an error passes outward through whatever contains it until something catches it or it reaches the prompt.

`let`

A special form: evaluates expressions in a new scope, with names bound in it first.

SYNTAX

```
(let {{name [value]} ...} body ...)
(let body)
```

ARGUMENTS

name

A symbol.

value

An expression: what the name is bound to. If it is left out, the name is bound to `NIL`.

body

An expression. It can also be written as a Q-expression, `{print x}`, which is then run as code.

RESULT

The value of the last *body* expression.

DESCRIPTION

1. A new scope is made, enclosed by the scope the `let` is in.

2. The bindings are taken in order. For each, *value* is evaluated **in the new scope**, and *name* is bound to the result there. A value can therefore use the names bound before it in the same `let`; a name bound twice keeps its last value.
3. The *body* expressions are evaluated in order, in the new scope. The values of all but the last are discarded.
4. The new scope is dropped, unless something made in it still refers to it: a function made by `fn` in the body keeps the scope, and can go on using its names after the `let` has returned (a closure).

A name bound by `let` hides one of the same name outside it for as long as the body runs, and the outer one is unchanged afterwards.

Inside the body, `set` binds or changes a name in the `let`'s own scope; `set!` changes a name in whichever scope it was made, which may be outside; and `def` always binds globally.

The last body expression is evaluated in tail position: a function whose body ends in a `let` that ends in a call to itself runs in constant stack, however many times it repeats.

With one argument only, that argument is the body: `(let body)` evaluates it in a new, empty scope. This means `(let {{a 1}})` does not bind anything; its one argument is run as code, and the result is the list `{a 1}`.

ERRORS

MESSAGE	CAUSE
`A 'let' binding is {name value}`	A binding that isn't a list, whose first element isn't a symbol, or that has more than two elements
`'let' supplied too few parameters`	No arguments at all

An error from evaluating a *value* or a *body* expression ends the `let` at once, and is its result.

SEE ALSO

`def`, `set`, `set!`, `fn`, `do`.

EXAMPLE

```
hylang> (let {{x 2} {y (+ x 1)}} (list x y))
=> {2 3}
hylang> (def {x} 10)
=> NIL
hylang> (let {{x 1}} (set {x} 5) x)
=> 5
hylang> x
=> 10
hylang> (def {add5} (let {{n 5}} (fn {a} {+ a n})))
=> NIL
hylang> (add5 1)
=> 6
hylang> n
=> Error: Unbound Symbol 'n'
hylang> (let {a 1} a)
=> Error: A 'let' binding is {name value}
```